RaceFlow

Document de passation | Etat d'avancement du projet F1

Mustapha HAMDAOUI ALAOUI | EMSI Rabat | 5 novembre 2026
Depot : github.com/Totito3GG/F1-Project

1. Resume

RaceFlow est un pipeline Data Engineering + ML qui recupere les resultats de Formule 1 (API Jolpica, remplacante d'Ergast), les fait passer par Kafka et Spark dans un Data Lake Bronze / Silver / Gold, puis predit le vainqueur d'un Grand Prix avec XGBoost. **Les etapes 1 a 6 sont terminees et testees** : les donnees 2015-2025 sont ingerees, nettoyees et transformees en features pretes pour le Machine Learning. Le code de l'etape 7 (ML) est pret mais n'a pas encore ete execute.

2. Etat d'avancement

#	Etape	Fichier(s)	Statut	Resultat
1	Structure du projet	README, .gitignore, requirements	Fait	Repo initialise
2	Kafka + Kafka UI	docker-compose.yml	Fait	localhost:8080
3	Producer API -> Kafka	ingestion/producer_results.py	Fait	8917 messages
-	Tests unitaires + CI	tests/, pytest.ini, ci.yml	Fait	3 tests OK, CI verte
4	Kafka -> Bronze (Spark)	streaming/kafka_to_bronze.py	Fait	8917 lignes Parquet
5	Bronze -> Silver	streaming/bronze_to_silver.py	Fait	4698 lignes, 0 doublon
6	Silver -> Gold (features)	streaming/silver_to_gold.py	Fait	4698 lignes, 19 col.
7	ML XGBoost + MLflow	ml/train.py	Code pret	Voir section 6
8	Orchestration Airflow	airflow/dags/f1_pipeline.py	A faire	
9	Dashboard Streamlit	dashboard/app.py	A faire	
10	Producer OpenF1 (tours)	ingestion/producer_live.py	A faire	Optionnel

3. Architecture actuelle

```
Jolpica API (2015-2025)
      |  ingestion/producer_results.py   (PC, venv Python)
      v
Kafka  topic f1.results  (Docker, localhost:9092 / kafka:29092)
      |  streaming/kafka_to_bronze.py    (conteneur spark)
      v
data/bronze/results   JSON brut + metadonnees Kafka, partition ingestion_date
      |  streaming/bronze_to_silver.py
      v
data/silver/results   colonnes typees, dedup par cle, partition season
      |  streaming/silver_to_gold.py
      v
data/gold/features    10 features + cible is_winner, partition season
      |  ml/train.py  (A FAIRE)
      v
XGBoost + MLflow  ->  Streamlit  (A FAIRE)        Airflow orchestre tout (A FAIRE)
```

Choix importants a connaitre

- **Cle de message** *saison-course-pilote* (ex. 2024-5-max_verstappen) : sert a dedupliquer en Silver. Kafka contient volontairement des doublons (2015-2024 envoyes deux fois) : Bronze = 8917, Silver = 4698.

- **Bronze ne parse pas le JSON** : la donnee brute reste intacte si l'API change.
- **Checkpoint Spark** (data/checkpoints/) : relancer kafka_to_bronze ne relit pas les memes messages.
- **Silver et Gold en mode overwrite** : on peut les relancer autant de fois qu'on veut (idempotence).
- **Pas de fuite de donnees** : toutes les fenetres Gold s'arretent a la course precedente (rowsBetween(-5, -1)). Verifie sur Verstappen 2025.
- **Spark tourne dans Docker**, pas sur Windows : rien a installer a part Docker Desktop.

4. Features de la table Gold

Feature	Signification
grid / pit_lane_start	Position de depart / depart des stands (grid = 0)
drv_avg_pos_last5	Position moyenne du pilote sur ses 5 dernieres courses
drv_wins_last5 / drv_podiums_last5	Victoires / podiums sur les 5 dernieres courses
drv_finish_rate_last10	% de courses terminees sur les 10 dernieres (fiabilite)
drv_points_season	Points du pilote dans la saison avant la course
drv_circuit_avg_pos	Position moyenne du pilote sur ce circuit par le passe
cons_avg_pos_last5 / cons_points_last5	Performance de l'ecurie sur les 5 dernieres courses
is_winner (cible)	1 si le pilote a gagne la course

5. Installation pour la personne suivante

Le dossier **data/** n'est pas sur GitHub (exclu par .gitignore). Il faut regenerer les donnees en relancant le pipeline dans l'ordre. Prevoir environ 15 minutes. Prerequis : Git, Python 3.11+, Docker Desktop lance.

a) Recuperer le projet et l'environnement Python

```
git clone https://github.com/Totito3GG/F1-Project.git
cd F1-Project
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # si bloque : Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
pip install -r requirements.txt
python -m pytest                       # attendu : 3 passed
```

b) Lancer Kafka et Spark

```
docker compose up -d
docker ps                              # attendu : kafka, kafka-ui, spark
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --topic f1.results --bootstrap-server localhost:9092
docker exec kafka /opt/kafka/bin/kafka-topics.sh --create --topic f1.laps --bootstrap-server localhost:9092
```

c) Faire tourner le pipeline (dans cet ordre)

```
python ingestion/producer_results.py --start 2015 --end 2025

docker exec spark /opt/spark/bin/spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.1 \
    --conf spark.jars.ivy=/tmp/.ivy2 /opt/app/streaming/kafka_to_bronze.py      # -> Bronze OK
docker exec spark /opt/spark/bin/spark-submit /opt/app/streaming/bronze_to_silver.py   # -> Silver OK
docker exec spark /opt/spark/bin/spark-submit /opt/app/streaming/silver_to_gold.py     # -> Gold OK
```

Sous PowerShell, mettre la commande kafka_to_bronze sur une seule ligne (sans le \).

d) Verifier

```
python -c "import pandas as pd; print(len(pd.read_parquet('data/gold/features')))"   # attendu : 4698
Kafka UI : http://localhost:8080
```

6. Ce qu'il reste a faire

Etape 7 : Machine Learning (prochaine etape)

• Ajouter a requirements.txt : `xgboost>=2.1`, `scikit-learn>=1.5`, `mlflow>=2.17`, puis `pip install -r requirements.txt`.

• Ajouter `mlflow.db` dans .gitignore.

• Coller le code de ml/train.py (fourni par Mustapha) puis lancer `python ml/train.py`.

• Split temporel : entrainement 2015-2023, test 2024-2025. Jamais de split aleatoire.

• Metrique principale : **winner_accuracy** (% de courses ou le favori du modele a gagne) a comparer a **pole_baseline_accuracy** (toujours parier sur le poleman). Le modele doit battre la baseline.

• Voir les runs : `mlflow ui --backend-store-uri sqlite:///mlflow.db` puis http://localhost:5000.

• Pistes d'amelioration : tester LightGBM, ajuster max_depth / learning_rate, ajouter des features (qualifications, meteo).

Etape 8 : Airflow

• Ajouter Airflow dans docker-compose.yml.

• DAG f1_pipeline : producer -> kafka_to_bronze -> bronze_to_silver -> silver_to_gold -> train, planifie apres chaque Grand Prix.

• Retries et alerte en cas d'echec.

Etape 9 : Dashboard Streamlit

• Lire data/gold/features et le modele MLflow.

• Afficher : probabilite de victoire par pilote pour une course, comparaison prediction / resultat reel, stats pilotes et ecuries.

Finitions

• Producer OpenF1 (ingestion/producer_live.py) pour les tours en temps reel (optionnel).

• Tests pour les jobs Spark, mise a jour du README (cocher les etapes, capture du dashboard), video demo de 2 minutes.

7. Checklist avant de passer le projet au suivant

[ ] Tout le code est enregistre (pas de point blanc sur les onglets VS Code).

[ ] `python -m pytest` passe en local.

[ ] `git status` est propre, puis `git push` fait.

[ ] La CI GitHub Actions est verte.

[ ] Le README est a jour : etapes cochees dans la section Avancement.

[ ] Ce document de passation est mis a jour (statut des etapes).

[ ] Le suivant sait expliquer son etape en 2 minutes : quoi, pourquoi, comment le verifier.